Cryptographic Hash Functions in Practice

From the birthday bound to HMAC, password storage, and real collisions

Diellart Mulolli (230756200035)

Code and LaTeX source: https://github.com/Diellart-Mulolli/hash-lab

October 20, 2026

Introduction

A cryptographic hash function compresses a message of any length into a short, fixed-length digest and is the quiet workhorse of modern security: it protects stored passwords, fingerprints software downloads, forms the core of HMAC and of every digital signature, and chains the blocks of a blockchain. Its usefulness rests on three properties — preimage, second-preimage and collision resistance — and on the fact that a well-designed n-bit hash costs about 2^n work to invert but only $2^{n/2}$ to collide, the birthday bound. This report works through the mathematics on a toy checksum, surveys the real status of MD5 through SHA-3, shows a length-extension forgery and HMAC by hand, estimates password-cracking and Merkle-tree costs, analyses a documented real-world failure (the 2017 SHAttered SHA-1 collision), and finally chooses hash-based mechanisms for a long-lived university system. Six small Python scripts were used to verify every calculation; they are published, with the LaTeX source and a running guide, at https://github.com/Diellart-Mulolli/hash-lab. Appendix A lists the values chosen in each script and its exact output.

Foundations

Question 1 — Security properties, and why a checksum is not a hash

(a) The three properties, in plain terms:

- **Preimage resistance (one-way).** Given a digest y, it is infeasible to find any x with $h(x) = y$. This protects *stored passwords*: a server keeps h(password), and an attacker who steals the database still cannot recover the passwords by inverting the hash.
- **Second-preimage resistance.** Given a *fixed* x, it is infeasible to find a different x' with $h(x') = h(x)$. This protects a *published software fingerprint*: the vendor pins the hash of one specific installer, and an attacker cannot craft a different file with the same hash.
- **Collision resistance.** It is infeasible to find *any* pair $x \neq x'$ with $h(x) = h(x')$. This protects *a CA-signed certificate or any digital signature*: the signer signs $h(M)$, so if the attacker can find two messages with the same hash (one benign, one rogue), a signature on the benign one is also valid on the rogue one.

Collision resistance is hardest, because the attacker may choose *both* messages and so enjoys the birthday effect: a collision costs only $\approx 2^{n/2}$ work, against $\approx 2^n$ for a (second-)preimage, where the attacker is pinned to one target. This is exactly why collisions fell first for MD5 and SHA-1 while preimages did not.

(b) Toy hash $h(X) = (3X_0 + 2X_1 + X_2) \bmod 256$, verified by `q1_toy_hash.py` (Appendix A.1).

(i) BID $= (66, 73, 68)$, so $h(\text{BID}) = (3 \cdot 66 + 2 \cdot 73 + 68) \bmod 256 = 412 \bmod 256 = \boxed{156}$.

(ii) *Second preimage:* add 1 to X_0 (which adds 3) and subtract 3 from X_2. This gives $(67, 73, 65) =$ `"CIA"` with $h = 156$. (The script confirms $256^2 = 65536$ messages share this hash.)

(iii) *Targeted preimage starting `OK`:* with $X_0 = 79$, $X_1 = 75$ fixed, solve for X_2: $X_2 = (156 - 3 \cdot 79 - 2 \cdot 75) \bmod 256 = 25$. So $(79, 75, 25)$ hashes to 156.

(iv) Both (ii) and (iii) are easy because the formula is **linear (affine) and invertible modulo** 256: each output bit depends smoothly on the input, there is no diffusion or one-wayness, and one byte can always be solved for from a single linear equation. The short 8-bit output is not the point — even a long linear checksum would fall the same way.

(c) With $P(k) \approx 1 - e^{-k(k-1)/2^{n+1}}$ for $n = 8$ (`q2_birthday.py`, Appendix A.2):

$$P(16) = 1 - e^{-16 \cdot 15/2^9} = 1 - e^{-240/512} \approx 0.374 \ (37.4\%).$$

The rule of thumb $k \approx 1.18 \cdot 2^{n/2} = 1.18 \cdot 16 \approx 18.9$ gives $P \approx \frac{1}{2}$; the exact smallest k with $P \geq 0.5$ from the formula is 20. A hash output must be **about twice as long as a symmetric key** for equal security because the best generic attack on the hash is the birthday collision at $2^{n/2}$, whereas brute-forcing a κ-bit key costs 2^κ; matching them needs $n/2 = \kappa$, i.e. $n = 2\kappa$ (so 256-bit SHA-256 pairs with 128-bit AES).

(d) A CRC-32 is a *linear* function over GF(2): $\mathrm{CRC}(a \oplus b) = \mathrm{CRC}(a) \oplus \mathrm{CRC}(b)$ (up to fixed constants). A stream cipher also encrypts by XOR with a keystream. So an attacker who flips chosen plaintext bits knows *exactly* which CRC bits flip, and can flip the corresponding bits of the encrypted CRC to keep it consistent — all without the key and without decrypting. The checksum therefore detects random noise but not an intelligent attacker; this is the WEP integrity flaw (Stamp, Section 5.5). Integrity needs a *keyed* MAC, not a linear checksum.

Question 2 — Output length and the status of real hash functions

(a) Output lengths and current status (collision figures from the cited cryptanalysis):

Hash	Output	Construction	Generic collision	Best known collision attack (year, cost)	Status today
MD5	128	Merkle–Damgård	2^{64}	identical-prefix in seconds ($\leq 2^{24}$); chosen-prefix $\approx 2^{39}$ (2009) [4]	Broken; forbidden
SHA-1	160	Merkle–Damgård	2^{80}	identical-prefix $2^{63.1}$ (2017, SHAttered) [2]; chosen-prefix $2^{63.4}$ (2020) [3]	Broken; retired by 2030 [7]
SHA-256	256	Merkle–Damgård	2^{128}	31 of 64 steps practical (2024) [5]; full function unbroken	Secure
SHA-512	512	Merkle–Damgård	2^{256}	31 of 80 steps, $2^{97.3}$ (2024) [5]; full function unbroken	Secure
SHA3-256	256	Sponge (Keccak)	2^{128}	5 of 24 rounds practical [6]; full function unbroken	Secure

Table 1: Hash status. "Steps/rounds" attacks are on round-reduced versions; the standard functions are intact.

(b) At 10^{12} hashes/s (`q2_birthday.py`): 2^{64} takes $\approx 1.8 \cdot 10^7$ s $\approx$ **214 days**; 2^{80} takes $\approx \mathbf{3.8 \cdot 10^4}$ **years**; 2^{128} takes $\approx \mathbf{1.1 \cdot 10^{19}}$ **years** (far beyond the age of the universe). The generic collision cost of SHA-1 is $2^{80} \approx 38{,}000$ years at this rate, yet the first real SHA-1 collision (2017) cost only $\approx 2^{63.1}$ compression calls — about 6,500 CPU-years plus 110 GPU-years, i.e. a large but affordable GPU budget. The difference is the whole point: cryptanalysis cut the cost by a factor of $\approx 2^{80-63.1} \approx 2^{17}$ ($\sim$130,000$\times$), so the "generic 2^{80}" figure stopped describing SHA-1 once the function was broken.

(c) In the birthday attack on signatures (Stamp, Section 5.4), Trudy prepares about $2^{n/2}$ innocuous variants of a "good" message and $2^{n/2}$ variants of an "evil" message; by the birthday bound a *good* variant and an *evil* variant are likely to share a hash. She has Alice sign the good variant, and the same signature is valid on the colliding evil one — so she needs only $2^{n/2}$ variants of each, not 2^n. A contract with 32 independent either/or phrasings yields 2^{32} variants, which defeats any hash with $n \leq 64$ bits, since $2^{64/2} = 2^{32}$. *Link to last week:* ECDSA signs $h(M)$, not M. P-256 gives ≈ 128-bit security, so it is paired with SHA-256 whose 2^{128} collision resistance matches it and is not the weak link. If MD5 were used instead, Trudy could collide a benign and a malicious contract and transfer Alice's valid P-256 signature onto the malicious one; the strong curve would not help, because the signature only ever commits to the hash.

(d) *Grover's* algorithm searches an unstructured space quadratically faster, so it lowers preimage resistance from 2^n to $\approx 2^{n/2}$ quantum evaluations. The *Brassard–Høyer–Tapp (BHT)* algorithm lowers collision search from $2^{n/2}$ to $\approx 2^{n/3}$, but only with $\approx 2^{n/3}$ quantum memory, which makes it uncompetitive with classical parallel collision search in practice. SHA-256 and SHA3-256 stay adequate because 256 bits leaves Grover preimage resistance at 2^{128} and BHT collision resistance at $\approx 2^{85}$ (and higher in realistic models) — still $\geq$ 128-bit post-quantum security. The hash-based post-quantum signature standards are **SLH-DSA** (FIPS 205, the SPHINCS$^+$ family) [9] and the stateful **LMS/XMSS** (NIST SP 800-208) [10]. Their security rests *only* on the (second-)preimage resistance of the underlying hash — no number-theoretic assumption like factoring or discrete logs — so they remain safe as long as the hash does, which is exactly what makes them conservative choices for the quantum era.

Constructions and Message Authentication

Question 3 — Inside the hash: Merkle–Damgård, the sponge, and length extension

(a) In **Merkle–Damgård** (MD5, SHA-1, SHA-2) the message is padded (a 1 bit, zeros, and the 64-bit length) and split into blocks $B_1, B_2, \ldots$; a compression function F updates a chaining value from a fixed constant A: $h = F(F(A, B_1), B_2)$ (Stamp, eq. 5.1). The output *is the entire final chaining value*, so the digest reveals the full internal state. In the **sponge** (SHA-3/Keccak) a 1600-bit state splits into a rate part ($r = 1088$ for SHA3-256) and a capacity part ($c = 512$); each block is XORed into the rate and the permutation f is applied (absorbing), then the output is read from the rate (squeezing). The output is only part of the state: the *capacity is never output.* This difference is decisive for the next part.

(b) Length-extension forgery against tag = SHA-256($K\|M$), demonstrated end-to-end in `q3_length_extension.py` (Appendix A.3; a from-scratch SHA-256 that can resume from a captured digest, cross-checked against `hashlib`).

(i) *What Mallory needs:* the message M, its tag, the desired suffix `&to=trudy`, and the *length* of K (16 bytes here). She does **not** need K. Because the digest equals the full chaining value after $K\|M\|$pad, she can set that state and keep hashing.

(ii) *Padding:* $K\|M$ is $16 + 18 = 34$ bytes. SHA-256 appends `0x80`, then zero bytes, then the 64-bit big-endian bit length $34 \cdot 8 = 272 =$ `0x110`, which is 30 bytes of "glue" padding, filling the first 64-byte block. This glue must appear *inside* Mallory's forged message, because the server computes SHA-256($K\|M'$) and only reproduces the captured state if M' literally contains those glue bytes that the original hash already absorbed.

(iii) *Why the server accepts:* the server computes SHA-256($K\|M\|$glue$\|$`&to=trudy`). Since the tag is exactly the state after the first block $K\|M\|$glue, resuming from the tag and absorbing the suffix yields the same digest. The script's forged tag equals the server's recomputed tag (`6abffc5f...7723c1ac`), and because the server uses the *last* `to=` value, the request now pays `trudy`.

(iv) *Unknown key length:* Mallory simply tries $|K| = 0, 1, 2, \ldots$; each guess is one candidate forged request, and she knows she succeeded when the server accepts one. This is a tiny search (a few dozen tries).

(c) The attack fails for: **SHA3-256**, because the sponge never outputs the capacity, so the digest does not reveal the state needed to resume; **SHA-512/256**, because it truncates its output, so the 256-bit digest exposes only half the internal state and cannot be used as a resumable chaining value; and **HMAC-SHA-256**, because the tag is $h((K \oplus opad) \,\|\, h(\cdots))$ — an outer keyed hash wraps the inner one, so the tag is not the resumable state of any key-prefixed message. Stamp also warns (eq. 5.3) against $h(M\|K)$: there, if the attacker finds *any* collision $h(M) = h(M')$ (feasible for a broken hash, and found offline without the key), then $h(M\|K) = h(M'\|K)$ for every K, so the construction inherits the hash's collision weakness directly.

Question 4 — Hash, MAC, or signature: choosing and tracing integrity mechanisms

(a)

Property	Plain hash $h(M)$	HMAC-SHA-256	ECDSA P-256 / SHA-256
Key material	none	shared secret K	private key (signer), public key (verifier)
Who can create it	anyone	holders of K	holder of the private key only
Who can verify it	anyone	holders of K	anyone, with the public key
Detects random errors?	yes	yes	yes
Detects a deliberate active attacker?	**no** (attacker recomputes h)	yes	yes
Authenticates the origin?	no	yes (to key holders)	yes (to one specific signer)
Non-repudiation?	no	no (both sides share K)	yes (only the signer has the key)
One real use	file dedup, download integrity vs. noise	TLS record MAC, API request auth	code signing, X.509 certificates

(b) HMAC-SHA-256 with $K = \texttt{key} = \texttt{0x6B}\,\texttt{0x65}\,\texttt{0x79}$, structure verified against the standard library in `q4_hmac.py` (Appendix A.4), $B = 64$, $ipad = \texttt{0x36}$, $opad = \texttt{0x5C}$.

(i) K is shorter than B, so it is right-padded with zero bytes to **64 bytes**.

(ii) $K \oplus ipad$: first three bytes $\texttt{0x5D}\,\texttt{0x53}\,\texttt{0x4F}$ (e.g. $\texttt{0x6B} \oplus \texttt{0x36} = \texttt{0x5D}$), and the remaining 61 bytes are $\texttt{0x36}$ (since $\texttt{0x00} \oplus ipad = ipad$). $K \oplus opad$: first three bytes $\texttt{0x37}\,\texttt{0x39}\,\texttt{0x25}$, remaining 61 bytes $\texttt{0x5C}$.

(iii) $\mathrm{HMAC}(M, K) = \text{SHA-256}\big((K \oplus opad) \,\|\, \text{SHA-256}((K \oplus ipad) \,\|\, M)\big)$.

(iv) SHA-256 adds at least 9 bytes of padding, so a 1000-byte message needs $\lceil (1000 + 9)/64 \rceil = 16$ blocks. HMAC's inner hash covers $(K \oplus ipad) \,\|\, M$, one extra block, $\lceil (1064 + 9)/64 \rceil = 17$ blocks; the outer hash covers $(K \oplus opad)$ plus the 32-byte inner digest, $\lceil (96+9)/64 \rceil = 2$ blocks. Total $17 + 2 = \mathbf{19}$ blocks versus 16 for a plain hash: about two extra compressions, **negligible** overhead.

(c) A SHA-256 checksum on the download page protects only against *accidental* corruption — truncated or bit-rotted downloads — because anyone, including an attacker, can recompute h. It does **not** protect against a deliberate attacker who replaces the installer *and* the checksum on the same page (a server compromise or a man-in-the-middle rewrites both; the plain-hash column of (a) shows "authenticates origin: no"). A better mechanism is a **digital signature** (code signing with ECDSA/Ed25519 over SHA-256), with the vendor's public key distributed out-of-band in the OS or browser trust store: only the vendor holds the private key, so the attacker cannot forge a valid signature, and (a) shows this is the only column with origin authentication and non-repudiation under a key the attacker does not have.

(d) HMAC appears in **HKDF** (TLS 1.3 key derivation). HKDF-Extract computes $\mathrm{PRK} = \mathrm{HMAC}(\text{salt}, \text{ECDH secret})$ to condense the non-uniform shared secret into a uniformly random pseudorandom key, and HKDF-Expand chains HMAC calls to derive several independent keys bound to context labels. HMAC's contribution is a *keyed pseudorandom function* whose output is computationally indistinguishable from random, which is what lets one Diffie–Hellman secret safely produce many session keys. (In RFC 6979 the same HMAC-as-PRF property instead derives the

ECDSA nonce deterministically from the private key and $h(M)$, removing the need for a random number generator.)

Applications and Real-World Investigation

Question 5 — Hash functions in applications: passwords, sealed bids, Merkle trees

(a) Password space $36^8 \approx 2.82 \cdot 10^{12} \approx 2^{41}$, 10^6 users, verified in `q5_passwords.py` (Appendix A.6).

(i) *Unsalted SHA-256:* searching the whole space once costs $2.82 \cdot 10^{12}/10^{10} \approx 282\,\text{s} \approx$ **4.7 minutes**, and that single table breaks *all* users at once, because identical passwords hash identically and the table is precomputable (rainbow tables).

(ii) *Per-user random salt, still SHA-256:* one chosen user still takes ≈ 4.7 minutes (the hash is just as fast), but all 10^6 users now cost $10^6\times$ that $\approx$ **8.9 years**, because no table is shared across users.

(iii) *Salted Argon2id* ($\approx 10^4$ guesses/s): one chosen user costs $2.82 \cdot 10^{12}/10^4 \approx$ **8.9 years** — a $10^6\times$ slowdown versus the fast hash.

A **salt** (stored, per-user, random) defeats precomputation and stops one effort from cracking every user; a **pepper** (a site-wide secret kept outside the database, e.g. in an HSM) adds a secret the attacker lacks even after stealing the database. **Memory-hard** functions force each guess to use a large amount of RAM, which removes the massive parallelism of GPUs and ASICs (they have little fast memory per core). Storing *encrypted* passwords is worse than hashing, because encryption is reversible: if the key leaks — often from the same server — every password is instantly recovered, whereas a hash is one-way. Current guidance: OWASP recommends **Argon2id** with $\geq$ 19 MiB memory, iteration count 2, parallelism 1 (or bcrypt work factor ≥ 10, or PBKDF2-HMAC- SHA-256 with $\geq$ 600,000 iterations) [11]; NIST SP 800-63B requires a salt of at least 32 bits and storage with an approved key-derivation function, with an optional secret keyed hash [12]. (Note that an 8-character alphanumeric space is only 2^{41}, so even Argon2id merely buys years for *one* user — password *length* still matters most.)

(b) Sealed bids submitted as $h(\text{bid})$ for whole euro amounts 1 to 10,000. Chuck simply computes $h(1), h(2), \ldots, h(10{,}000)$ — at most $\mathbf{10^4}$ hash computations — and matches Alice's published value to read her bid before the deadline. The commitment is not *hiding* because the input space is tiny and low-entropy (a trivial dictionary attack). *Fix:* commit to $h(\text{bid} \,\|\, r)$ with a long random nonce r, revealed only at opening. Then *hiding* comes from **preimage resistance** plus the high entropy of r (the bid cannot be guessed), and *binding* comes from **collision/second-preimage resistance** (Alice cannot find a second (bid', r') with the same hash, so she cannot change her bid after committing).

(c) Merkle tree with $T(x, y) = (3x + 5y + 7) \bmod 101$, leaves L_1=23, L_2=8, L_3=61, L_4=40, verified in `q5_merkle.py` (Appendix A.5).

(i) $N_{12} = T(23, 8) = 116 \bmod 101 = 15$; $N_{34} = T(61, 40) = 390 \bmod 101 = 87$; $R = T(15, 87) = 487 \bmod 101 = \boxed{83}$.

(ii) Inclusion proof for transaction 3: L_3 is the *left* child of N_{34}, so the proof is [$L_4 = 40$ (right sibling), $N_{12} = 15$ (left uncle)]. The light client recomputes $T(N_{12}, T(L_3, L_4)) = T(15, T(61, 40)) = T(15, 87) = 83 = R$ and accepts, having stored only the root.

(iii) If L_3 becomes 62: $N'_{34} = T(62, 40) = 90$ and $R' = T(15, 90) = \boxed{98} \neq 83$. The change is detected because the recomputed root no longer matches the trusted root.

(iv) A block of $4096 = 2^{12}$ transactions needs an inclusion proof of $\log_2 4096 =$ **12** hashes. The block header stores only the root (Stamp, Section 5.8.2), so a light client verifies any transaction with a 12-hash proof instead of the whole block; and because each block header also commits to the

previous block's hash, altering an old transaction changes its root, hence its header hash, hence every following block's "previous hash" — invalidating the entire chain after it.

Question 6 — SHAttered: the first SHA-1 collision (2017)

(a) Event profile.

Date(s)	Announced 23 February 2017 (SHA-1 deprecated by NIST since 2011).
Who	Researchers at Google and CWI Amsterdam: M. Stevens, E. Bursztein, P. Karpman, A. Albertini, Y. Markov [2].
Affected systems and scale	Everything relying on SHA-1 collision resistance: TLS/X.509 certificates (already being sunset), digital signatures, PGP, and content-addressed systems — Git and Subversion (the WebKit SVN repository was corrupted by committing the two PDFs). SHA-1 was one of the most widely deployed hashes.
Hash and property	SHA-1 (160-bit, Merkle–Damgård); identical-prefix *collision resistance*.
Identifier	shattered.io; CVE-2005-4900 (SHA-1 is not collision resistant).
Fix / outcome	Two PDFs with the same SHA-1 were published; migration to SHA-256 accelerated, Git shipped a collision-detecting SHA-1 (sha1dc) as the default in v2.13 (May 2017) [8], and browsers already rejected SHA-1 certificates (Chrome 56, Jan 2017).

(b) Technical root cause. SHA-1's compression function was broken by *differential* cryptanalysis: the researchers built a two-block differential path (using disturbance vectors) so that a carefully chosen difference between two inputs cancels out, producing the same chaining value. Combined with enormous GPU computation this cost $\approx 2^{63.1}$ compression calls — below the generic 2^{80} — and yielded two distinct PDF files with the *same SHA-1 digest*. The property that failed was **(identical-prefix) collision resistance**; preimage resistance was not broken. What the attacker gains is two colliding documents: a signature or certificate on one is valid on the other, and content-addressed stores that trust SHA-1 as an identifier can be poisoned. The *retirement timeline* was triggered step by step: Wang's 2005 theoretical attack (2^{69}) started deprecation; SHAttered (2017, identical-prefix, $2^{63.1}$) proved it practical; and "SHA-1 is a Shambles" (2020, *chosen*-prefix, $2^{63.4}$, $\sim$ \$45k) [3] let attackers pick both prefixes freely — enough to forge realistic certificates and PGP identities — which pushed NIST to announce full retirement of SHA-1 by 2030 [7].

(c) Classification. The **hash function itself was broken by cryptanalysis**: the collision exploited mathematical weaknesses in SHA-1's compression function, not a misuse by engineers. (This contrasts with, say, the LinkedIn breach, where a sound idea was spoiled by fast, unsalted hashing.) SHA-1 was used as intended — for collision resistance — and that property simply no longer held.

(d) Response and lessons. Vendors and standards bodies responded on several fronts: Git added collision detection by default within months [8], CAs and browsers had already scheduled the SHA-1 certificate sunset (2016–2017), and NIST set a hard 2030 end date [7]. The response was effective but *slow to start*: the community had warned since 2005, yet SHA-1 lingered for years, and only the 2020 chosen-prefix result made migration urgent. A concrete control that would have reduced the impact is **enforced crypto-agility with deprecation deadlines** — standards and software that make the hash a swappable parameter and bar weakened algorithms on a fixed schedule — together with collision detection enabled by default in content-addressed systems, so that a broken primitive can be retired quickly rather than over a decade.

Design Decisions

Question 7 — Choosing hash-based mechanisms for UniServices

(a) For 40,000 users over a 10-year lifetime:

Requirement	Mechanism and parameters	Why not the naive alternative	One pitfall to avoid
R1. Store passwords	Argon2id (19 MiB, $t = 2$, $p = 1$), 128-bit per-user salt, optional pepper in an HSM [11]	Plain/unsalted SHA-256 is fast and precomputable, so one GPU table breaks every account	Setting memory/iterations too low, or keeping the pepper on the same server as the hashes
R2. Authenticate API requests	HMAC-SHA-256 over the request, with a timestamp and nonce, keyed by the per-install shared secret	$h(K\|\|M)$ allows length-extension forgery; a plain hash authenticates nothing	No replay protection, or shipping the key in the app binary where it can be extracted
R3. Distribute exam-client updates	Code signing: ECDSA P-256 (or Ed25519) over SHA-256, public key pinned in the installed client	A SHA-256 checksum on the same server is rewritten by the same attacker who swaps the update	Leaving the signing key unprotected (sign in an HSM), or no rollback/version control
R4. Tamper-evident grade audit log	Hash chain / Merkle log: $H_i = \text{SHA-256}(H_{i-1}\|\|\text{entry}_i)$, with roots periodically signed and published for the auditor	A plain log on the same server can be silently rewritten by an administrator	Not anchoring the head externally: an insider could recompute the whole chain unless roots are signed/published

(b) Post-quantum outlook for each choice:

- **R1 (Argon2id):** no change — it is hash/symmetric-based, Grover only halves preimage resistance, and 256-bit output keeps $\geq$ 128-bit security.
- **R2 (HMAC-SHA-256):** no change — HMAC is symmetric and remains secure against quantum attackers at these sizes.
- **R3 (ECDSA code signing): must change** — Shor's algorithm breaks ECDSA, and a 10-year client with a long-lived trust anchor makes migration urgent; move to a hash-based or lattice signature (SLH-DSA/LMS or ML-DSA).
- **R4 (hash chain):** the hashing needs no change (SHA-256 is fine), but the *signature* on the published roots must migrate to a post-quantum scheme so the auditor's verification stays sound.

Conclusion

Hash functions underpin passwords, downloads, MACs, signatures and blockchains, and their security rests on three properties plus the birthday bound, which forces a hash to be twice as long as a symmetric key. The toy checksum showed why linearity and short output make a function trivially invertible, and the survey showed the real split in the field: MD5 and SHA-1 are broken by cryptanalysis and retired, while SHA-256, SHA-512 and SHA3-256 remain intact. The constructions matter as much as the function: Merkle–Damgård exposes its whole state and so allows length extension, which HMAC and the sponge avoid, and the SHAttered case confirmed that a mathematically broken primitive endangers every signature built on it. The applications reinforced that correct *use* is just as important as a strong function — salting and memory-hard hashing for passwords, high-entropy commitments for sealed bids, and signed Merkle roots for tamper-evidence. Finally, the post-quantum view is reassuring for hashing: 256-bit hashes survive Grover and BHT, and hash-based signatures

(SLH-DSA, LMS/XMSS) inherit that strength, so the real migration work for a long-lived system lies in its signatures, not its hashes.

Appendix A — Python Verification: Chosen Values and Outputs

The six scripts are in the repository https://github.com/Diellart-Mulolli/hash-lab (see its README for the running guide). They were run with Python 3.12.3 (standard library only). The outputs below are copied unchanged from the terminal, and the key results are cross-checked independently: the custom SHA-256 in A.3 is validated against Python's `hashlib`, and the HMAC structure in A.4 against Python's `hmac` module.

A.1 `q1_toy_hash.py` (Question 1(b))

Chosen values: toy hash $h(X) = (3X_0 + 2X_1 + X_2) \bmod 256$; message BID $= (66, 73, 68)$; targeted prefix `OK`$= (79, 75)$. Command: `python3 q1_toy_hash.py`

```
message BID = (66, 73, 68) = b'BID'
(i)   h(BID) = (3*66 + 2*73 + 68) mod 256 = 412 mod 256 = 156

(ii)  second preimage: (67, 73, 65) = b'CIA', h = 156  (equal: True)
      total 3-byte messages with this hash: 65536 (= 256^2, one for every (X0,X1))

(iii) targeted preimage starting 'OK': X2 = (156 - 3*79 - 2*75) mod 256 = 25
      message = (79, 75, 25) = b'OK\x19', h = 156  (equal: True)
```

Check. $h(\text{BID}) = 156$ matches the hand computation; the second preimage `CIA` and the targeted preimage both hash to 156; and a full scan confirms 65536 messages share the value, showing the formula is many-to-one and linear.

A.2 `q2_birthday.py` (Questions 1(c), 2(b), 2(c))

Chosen values: birthday formula $P(k) = 1 - e^{-k(k-1)/2^{n+1}}$; $n = 8$ with $k = 16$; attacker rate 10^{12}/s; exponents $64, 80, 128$; 32 either/or phrasings. Command: `python3 q2_birthday.py`

```
Q1(c): 8-bit toy hash
  P(16 messages) = 1 - exp(-16*15 / 2^9) = 1 - exp(-240/512) = 0.3742  (~37.4%)
  k for P~=50%: 1.18 * 2^(8/2) = 1.18 * 16 = 18.9  (~19 messages)
  smallest k with P>=0.5 (exact from the formula): 20

Q2(b): attacker at 1e12 hashes/second
  2^64: 1.84e+07 s = 214 days
  2^80: 1.21e+12 s = 3.83e+04 years
  2^128: 3.4e+26 s = 1.08e+19 years
  for comparison, 2^80 / 1e12 = 3.83e+04 years
  SHAttered (2017) used ~2^63.1 compression calls; 2^63.1/1e12 = 114.41 days at this imagined rate

Q2(c): birthday attack on signatures
  32 independent either/or phrasings -> 2^32 = 4294967296 variants of each message
  collision between the 'good' and 'evil' sets needs ~2^(n/2) variants;
  2^32 variants defeat any hash with n <= 64 bits (2^(64/2) = 2^32).
```

Check. $P(16) \approx 37.4\%$ and the 50% threshold near $k \approx 19$ (rule of thumb) / 20 (exact) match Question 1(c); the $2^{64}, 2^{80}, 2^{128}$ times and the 2^{32} variant count match Questions 2(b)–(c).

A.3 `q3_length_extension.py` (Question 3(b))

Chosen values: 16-byte key `"sixteen-byte-key"`; M = `"amount=10&to=alice"`; appended suffix `"&to=trudy"`. The script implements SHA-256 from scratch so it can resume from a captured digest. Command: `python3 q3_length_extension.py`

```
sanity: custom SHA-256 matches hashlib: True
intercepted M   = b'amount=10&to=alice' (18 bytes)
intercepted tag = 2f54ff5373411d83d51cc5233569996c17d1e97323567152ff07ef6450cb449a
```

```
glue padding = 0x80 then zeros then 00..0110 (30 bytes; 0x110 = 272 bits)
forged M' = M || glue(30 B) || b'&to=trudy'  (total 57 bytes)
forged tag (no key)   = 6abffc5fdeeda4397a8d76ba769e43000aa94e933bee39ce881e3d7a7723c1ac
server tag (with key) = 6abffc5fdeeda4397a8d76ba769e43000aa94e933bee39ce881e3d7a7723c1ac
forgery accepted: True
server uses the LAST to= value, so the request now pays to=trudy.
```

Check. The from-scratch SHA-256 matches `hashlib` (first line), and the forged tag computed *without* the key equals the tag the server computes *with* the key — a successful length-extension forgery.

A.4 `q4_hmac.py` (Question 4(b))

Chosen values: $K = \texttt{key} = \texttt{0x6B 0x65 0x79}$; $B = 64$; $ipad = \texttt{0x36}$, $opad = \texttt{0x5C}$; 1000-byte message for the block count. Command: `python3 q4_hmac.py`

```
key K = b'key' = 0x6B 0x65 0x79

(i)   K is shorter than 64 bytes, so it is right-padded with zero bytes to 64 bytes.

(ii)  first three bytes (the rest are the pad byte itself, since 0x00 XOR pad = pad):
  K XOR ipad: 0x5D 0x53 0x4F ... then 0x36 x61
  K XOR opad: 0x37 0x39 0x25 ... then 0x5C x61
    ipad: k=0x6B^0x36=0x5D ; e=0x65^0x36=0x53 ; y=0x79^0x36=0x4F
    opad: k=0x6B^0x5C=0x37 ; e=0x65^0x5C=0x39 ; y=0x79^0x5C=0x25

(iii) HMAC(M,K) = SHA256( (K XOR opad) || SHA256( (K XOR ipad) || M ) )

(iv)  SHA-256 of a 1000-byte message: ceil((1000+9)/64) = 16 blocks
      HMAC inner SHA-256 over (K^ipad)||M: ceil((1064+9)/64) = 17 blocks
      HMAC outer SHA-256 over (K^opad)||digest: ceil((96+9)/64) = 2 blocks
      HMAC total = 17 + 2 = 19 blocks vs 16 for a plain hash (+3 blocks, ~2 extra compressions, negligible).

cross-check on M=b'transfer 100 EUR to Bob': manual == hmac.new: True
  HMAC = ca9c97583730acf2db3b30015cfcdb15b70ae3aace136d7ead2d43050b95d2e3
```

Check. The padded key length, the first bytes of $K \oplus ipad / K \oplus opad$, and the block counts match Question 4(b); the manual two-pass construction equals Python's `hmac.new`.

A.5 `q5_merkle.py` (Question 5(c))

Chosen values: combiner $T(x, y) = (3x + 5y + 7) \bmod 101$; leaves $(23, 8, 61, 40)$; altered leaf $61 \to 62$. Command: `python3 q5_merkle.py`

```
(i) building the tree
  N12 = T(23, 8)  = (3*23 + 5*8  + 7) mod 101 = 116 mod 101 = 15
  N34 = T(61, 40) = (3*61 + 5*40 + 7) mod 101 = 390 mod 101 = 87
  R   = T(N12,N34)= (3*15 + 5*87 + 7) mod 101 = 487 mod 101 = 83

(ii) inclusion proof for transaction 3 (leaf L3, which is the LEFT child of N34)
  proof = [ L4 = 40 (sibling, right), N12 = 15 (uncle, left) ]
  light client: T(N12, T(L3, L4)) = T(15, T(61,40)) = T(15,87) = 83  (matches root 83: True)

(iii) transaction 3 altered: L3 = 61 -> 62
  N34' = T(62, 40) = 90,  R' = T(15, 90) = 98
  root changed 83 -> 98: detected because the stored/signed root no longer matches (True)

(iv) proof size for a block of 4096 transactions
  depth = log2(4096) = 12, so an inclusion proof has 12 hashes
```

Check. Root $R = 83$; the inclusion proof for transaction 3 recomputes to 83; the altered leaf changes the root to 98; and a 4096-leaf tree needs 12 proof hashes — matching Question 5(c).

A.6 `q5_passwords.py` (Question 5(a))

Chosen values: charset 36, length 8 (36^8 space); 10^6 users; 10^{10}/s fast hash, 10^4/s Argon2id. Command: `python3 q5_passwords.py`

```
password space = 36^8 = 2,821,109,907,456 ~= 2^41
users = 1,000,000

(i) unsalted SHA-256:
    search the whole space once = 2,821,109,907,456/1e+10 = 0.0784 hours
    the same work breaks ALL users (identical passwords hash identically; precomputable).

(ii) per-user random salt, still SHA-256:
    one chosen user  = 0.0784 hours (same as (i): the hash is still fast)
    all 1,000,000 users = 1,000,000 x that = 8.94 years (no shared table any more)

(iii) salted Argon2id (~1e4 guesses/s):
    one chosen user = 2,821,109,907,456/1e+04 = 8.94 years
    slowdown vs fast hash = 1e+06x
```

Check. Unsalted SHA-256 falls in minutes for all users; salting makes the all-users cost ≈ 8.9 years; Argon2id makes even one user ≈ 8.9 years — the figures used in Question 5(a).

References

[1] M. Stamp, *Information Security: Principles and Practice*, 3rd ed. Hoboken, NJ, USA: Wiley, 2022, ch. 5.

[2] M. Stevens, E. Bursztein, P. Karpman, A. Albertini, and Y. Markov, "The first collision for full SHA-1," in *Advances in Cryptology – CRYPTO 2017* (LNCS 10401). Cham, Switzerland: Springer, 2017, pp. 570–596. [Online]. Available: https://shattered.io

[3] G. Leurent and T. Peyrin, "SHA-1 is a shambles: First chosen-prefix collision on SHA-1 and application to the PGP web of trust," in *Proc. 29th USENIX Security Symp.*, 2020, pp. 1839–1856.

[4] M. Stevens, A. Lenstra, and B. de Weger, "Chosen-prefix collisions for MD5 and applications," *Int. J. Appl. Cryptogr.*, vol. 2, no. 4, pp. 322–359, 2012.

[5] Y. Li, F. Liu, and G. Wang, "New records in collision attacks on SHA-2," in *Advances in Cryptology – EUROCRYPT 2024* (LNCS 14651). Cham, Switzerland: Springer, 2024, pp. 158–186, Cryptology ePrint Archive, Paper 2024/349. [Online]. Available: https://eprint.iacr.org/2024/349

[6] J. Guo, G. Liao, G. Liu, M. Liu, K. Qiao, and L. Song, "Practical collision attacks against round-reduced SHA-3," *J. Cryptology*, vol. 33, no. 1, pp. 228–270, 2020, doi: 10.1007/s00145-019-09313-3.

[7] NIST, "NIST retires SHA-1 cryptographic algorithm," News, 15 Dec. 2022. [Online]. Available: https://www.nist.gov/news-events/news/2022/12/nist-retires-sha-1-cryptographic-algorithm (accessed Oct. 20, 2026).

[8] GitHub, "Git 2.13 has been released," The GitHub Blog, 10 May 2017. [Online]. Available: https://github.blog/open-source/git/git-2-13-has-been-released/ (accessed Oct. 20, 2026).

[9] *Stateless Hash-Based Digital Signature Standard*, FIPS PUB 205, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Aug. 2024, doi: 10.6028/NIST.FIPS.205.

[10] D. Cooper *et al.*, "Recommendation for stateful hash-based signature schemes," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST SP 800-208, Oct. 2020, doi: 10.6028/NIST.SP.800-208.

[11] OWASP, "Password Storage Cheat Sheet," OWASP Cheat Sheet Series, 2024. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (accessed Oct. 20, 2026).

[12] D. Temoshok *et al.*, "Digital identity guidelines: Authentication and authenticator management," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST SP 800-63B, 2025, doi: 10.6028/NIST.SP.800-63b.

[13] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," Internet Eng. Task Force, RFC 2104, Feb. 1997. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2104

[14] *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*, FIPS PUB 202, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Aug. 2015, doi: 10.6028/NIST.FIPS.202.